

Backlog

Smart Campus Assistant

מגיש: יהלי באבאני
מרץ 2026

שלב א': ניתוח מערכת (Analysis)

1. אפיון דרישות וכתיבת מסמך SRS

- **User Story**: כמנתח מערכת, אני רוצה לאפיין את דרישות המוצר (פונקציונליות ולא-פונקציונליות), כדי להבטיח שהמערכת תיתן מענה מדויק ויעיל לצרכי הסטודנטים בקמפוס.
- **עדיפות**: גבוהה.
- **הערכת זמן**: כ-4 שעות.
- **Definition of Done**:
 - נכתב מסמך SRS מקיף ומאושר.
 - המסמך כולל הגדרת דרישות לא-פונקציונליות ברורות עם מדדים כמותיים (למשל: זמני תגובה, מגבלות עומס).
 - הוגדרו ופורטו לפחות 5 תרחישי שימוש (Use Cases) שונים ומרכזיים של סטודנטים ומנהלים.

2. מידול חזותי (UML)

- **User Story**: כמנתח מערכת, אני רוצה ליצור תרשימים חזותיים של זרימת העבודה, כדי שהמפתחים יבינו בדיוק כיצד השחקנים מתקשרים עם המערכת מאחורי הקלעים.
- **עדיפות**: גבוהה.
- **הערכת זמן**: כ-3 שעות.
- **Definition of Done**:
 - נוצרה דיאגרמת Use Case הממפה את כל השחקנים במערכת (סטודנט, מנהל, API) ואת הפעולות האפשריות שלהם.
 - נוצרה דיאגרמת Sequence המציגה את זרימת הנתונים המדויקת של בקשה (החל מהקלדת השאלה ב-Frontend ועד קבלת תשובת ה-AI).

שלב ב': ניהול פרויקט (Management)

1. תכנון העבודה והקמת Backlog

- **User Story:** כמנהל פרויקט, אני רוצה לפרק את דרישות המערכת למשימות ברורות ומתועדפות, כדי להבטיח עמידה בלוח הזמנים הלחץ של האקטון.
- **עדיפות:** גבוהה - תנאי מקדים לתחילת הפיתוח.
- **הערכת זמן:** כ-2 שעות.
- **Definition of Done:**
 - הוקם Backlog מפורט המחולק לשלבי הפרויקט השונים (אפיון, ניהול, פיתוח, תשתיות).
 - כל משימה הוגדרה במבנה אחיד וברור הכולל תעדוף והערכת זמנים.

2. מעקב יומי והפקת לקחים (Retrospective)

- **User Story:** כמנהל פרויקט, אני רוצה לעקוב אחר ההתקדמות היומית ולנתח את תהליך העבודה, כדי לזהות חסימות בזמן אמת ולהסיק מסקנות להמשך.
- **עדיפות:** בינונית - משימה מתמשכת לאורך כל הפרויקט.
- **הערכת זמן:** כשעה ביום.
- **Definition of Done:**
 - קיים יומן עבודה יומי ומלא הכולל מעקב שוטף: מה בוצע היום, אילו חסימות היו ומה התכנון למחר.
 - נכתב דו"ח רטרוספקטיבה (Retrospective) מסכם המנתח תובנות מרכזיות והפקת לקחים מהתהליך כולו.

שלב ג': פיתוח המערכת (AI & Development)

1. תשתית שרת ומסד נתונים למידע קמפוזי (Backend Core)

- **User Story:** בתור מפתח המערכת, אני רוצה להקים מסד נתונים ושרת עורפי המכילים מידע מובנה שמזן מראש על חדרים, שעות קבלה, מבחנים ושאלות נפוצות, כדי שיהיה מקור מאומת שממנו הבוט ישלף נתונים.
- **עדיפות:** גבוהה מאוד - דרישת המינימום הבסיסית ביותר (התשתית).
- **הערכת זמן:** כ-6 שעות.
- **Definition of Done:**
 - סכמות הנתונים למידע הקמפוזי מוגדרות ועובדות.
 - מסד הנתונים מכיל נתוני התחלה (Seed data) שהוזנו מראש.
 - נתיבי ה-API לשליפת המידע קיימים וניתנים לבדיקה.

2. שירות בינה מלאכותית וסיווג שאלות (AI Service)

- **User Story:** בתור סטודנט, אני רוצה לשאול שאלות חופשיות ולקבל תשובות מיידיות ומדויקות, כדי שלא אצטרך לחפש מידע ידנית באתר המכללה.
- **עדיפות:** גבוהה מאוד - ליבת הפונקציונליות של המוצר המוגדר.
- **הערכת זמן:** כ-8 שעות.
- **Definition of Done:**
 - נתיב השרת מקבל שאלת משתמש ומאתר את ההקשר (Context) הרלוונטי ממסד הנתונים.
 - שירות ה-AI קורא את ההקשר ומסווג את השאלה לקטגוריה הנכונה (למשל: "מבחנים" או "שעות קבלה").
 - ה-AI מחזיר תשובה טקסטואלית ברורה ואמינה המבוססת על נתוני הקמפוס בלבד.

3. ממשק משתמש אינטרנטי (Frontend Chat App)

- **User Story:** בתור סטודנט, אני רוצה אפליקציית רשת קלת-משקל ונוחה, כדי שאוכל להקליד בה את השאלות שלי ולראות את התשובות בצורה של צ'אט.
- **עדיפות:** גבוהה מאוד - הנגשת המוצר למשתמש הקצה (דרישת ה-Frontend).
- **הערכת זמן:** כ-6 שעות.
- **Definition of Done:**
 - ממשק המשתמש עולה בדפדפן וכולל תיבת טקסט וכפתור שליחה.
 - יש חיווי ויזואלי (כמו "טוען...") בזמן שהמערכת ממתינה לתשובת ה-AI.
 - התשובה והסיווג שלה מופיעים על המסך בצורה ברורה וקריאה.

4. מערכת אבטחה, משתמשים והגבלת קצב (Auth & Rate Limit)

- **User Story:** בתור מנהל מערכת, אני רוצה להגביל את הגישה לצ'אט רק לסטודנטים רשומים ולהגביל את כמות השאלות, כדי למנוע הצפות, עומסים וניצול של ה-API Key.
- **עדיפות:** בינונית - תוספת חשובה לפרודקשן אך אינה חלק מהגדרת ה-MVP המקורית.
- **הערכת זמן:** כ-4 שעות.
- **Definition of Done:**
 - מסד הנתונים תומך במודל משתמשים וסיסמאות מוצפנות.
 - ממשק הצ'אט דורש התחברות ומנפיק טוקן (JWT).
 - השרת חוסם משתמשים ששואלים יותר מ-5 שאלות בדקה.

5. לוח מחוונים למנהלים (Admin Dashboard)

- **User Story:** בתור מנהל מערכת, אני רוצה פאנל ויזואלי לניהול המידע (CRUD), כדי שאוכל לעדכן את לוחות המבחנים ושעות הקבלה בקלות בלי לשנות קוד או לגעת ישירות במסד הנתונים.
- **עדיפות:** בינונית - הוסף כדי להפוך את המערכת לדינמית באמת.
- **הערכת זמן:** כ-6 שעות.
- **Definition of Done:**
 - קיים מסך ניהול נפרד המוגן בהרשאות מנהל בלבד.
 - מנהל יכול לצפות בטבלאות של כל המידע הקמפוזי והמשתמשים.
 - מנהל יכול להוסיף, לערוך ולמחוק מידע דרך חלונות שונים שמעדכנים את מסד הנתונים.

6. מודעות לזמן הנוכחי ב-AI (Time-Awareness)

- **User Story:** בתור סטודנט, אני רוצה לשאול "מה פתוח עכשיו" או "מתי המבחן הבא", כדי לקבל תשובה שמתחשבת בתאריך והשעה בזמן אמת.
- **עדיפות:** נמוכה - פיצ'ר חכם מתקדם לסוף הפיתוח.
- **הערכת זמן:** כ-2 שעות.
- **Definition of Done:**
 - השרת מזריק את הזמן הנוכחי לפרומפט הנסתר של הבוט.
 - בדיקה מוכיחה שהבוט מסוגל להשוות בין הזמן הנוכחי לבין זמני הפעילות במסד הנתונים ולענות בהתאם.

שלב ד': תשתיות ופריסה (Infrastructure)

1. קונטיינריזציה וניהול סודות (Docker)

- **User Story:** כמפתח DevOps, אני רוצה לארוז את סביבת ההרצה של השרת והמערכת, כדי לאפשר לחברי צוות או בודקים לפרוס את המערכת בפקודה אחת ובטוחה.
- **עדיפות:** גבוהה.
- **הערכת זמן:** כ-4 שעות.
- **Definition of Done:**
 - נכתבו קבצי Dockerfile מאופטמים.
 - נכתב קובץ `docker-compose.yml` המאפשר את הרצת כלל רכיבי המערכת בפקודה אחת.
 - יושם מנגנון מאובטח לניהול סודות (משתני סביבה ו-API Keys) שאינו חושף מידע רגיש.

2. איכות קוד, תיעוד ובדיקות

- **User Story:** כמפתח תוכנה, אני רוצה לוודא שהקוד נבדק, אמין ומתועד היטב במאגר משותף, כדי להבטיח תחזוקה קלה ושקיפות של הארכיטקטורה.
- **עדיפות:** בינונית.
- **הערכת זמן:** כ-5 שעות.
- **Definition of Done:**
 - קיים מאגר GitHub מסודר המשקף את כל שלבי הפיתוח.
 - נכתבו והופעלו בדיקות יחידה (Unit Tests) ויש הוכחה מוקלטת/מצולמת להרצת בדיקות תקינה.
 - המאגר כולל קובץ `README.md` מקיף המכיל הוראות התקנה ברורות, פירוט ארכיטקטורת המערכת והסבר על הפריסה.